

Development Report - Noé Auguste

I. Concept & Design Choices

Game overview

My game is, as asked, a first person 3D game in unity 6.3 focused on speed. I've always been interested in game movement with Team fortress 2 or quake with air strafing and bunny hopping. It inspired me to make this tag-like game.

I could have made this game multiplayer like I did for my EPITA's game last year but I feel like I would have spent too much time working only on the multiplayer and not discovering new things and we needed to make an AI so the idea of an AI chaser came naturally.

I also made a dither effect shader to make my game feel more unique and speed lines visual effect depending on the player speed to enhance the feeling of speed

Unique gameplay mechanics chosen

As mentioned before one key aspect of my game is the movement, mainly with the ability to air strafe to gain speed and bunny hop to maintain it. However, accumulating such speed was a problem in a small map and this is how I got the idea to try making a 3D procedural generation.

II. Technical Implementation

Core gameplay mechanics

Bunny Hopping

Rather than using Unity's default character controller movement, the system uses a custom acceleration model. When autoHop is enabled, the player can hold the jump button to automatically re-jump the moment they touch the ground, preserving horizontal momentum across jumps. This is achieved by only resetting vertical velocity on landing ($velocity.y = -1f$) without zeroing horizontal speed, allowing the player to chain jumps and build speed progressively.

Air Strafing

While airborne, movement is handled separately from ground movement. The AirAccelerate function caps the wish speed and uses a dot product between the current velocity and the desired direction to determine how much acceleration to add. This means the player cannot accelerate forward beyond the cap, but can gain speed

by moving the mouse while holding a strafe key. This creates the characteristic curved acceleration of Quake-style air strafing that I wanted.

Air Control

An additional `ApplyAirControl` function allows slight directional influence while airborne, but only when not strafing (because horizontal input is blocked). It works by rotating the horizontal velocity vector towards the wished direction proportionally to how aligned they already are, giving the player fine control when moving straight but not during strafing.

AI system

Unity's NavMesh system is usually designed for static geometry, however the generation of the level is built dynamically (during the runtime) which causes trouble baking the NavMesh Surface.

To solve this I used an Async runtime baking, each chunk bakes its own NavMesh asynchronously using `NavMeshBuilder.UpdateNavMeshDataAsync()`. When the chunk finishes generating its mesh collider `BakeNavMeshAsync()` is triggered via a coroutine, collecting the physics collider and baking navigation data. The coroutine waits frame by frame until the async operation completes, to try limiting any frame rate spike.

The chaser uses the companion script we've made in class. I only added the `OnTriggerEnter()` method to detect when the chaser caught the player.

Scriptable Objects usage

Each collectible item is defined by an `ItemData` Scriptable Object containing three fields : `itemName`, `affectedStat` and `attributeBoost`. The `affectedStat` field uses a `StatType` enum (`groundSpeed`, `airSpeed`, `JumpForce`) to identify which player attribute the item affects. `AttributeBoost` is the multiplier of the boost (by how much the stat is multiplied) This means adding a new item type to the game requires zero code changes that will be useful in the future if I want to add more interesting items.

The `Item` component reads `itemData` to know what to give the player when picked up. The `Inventory` stores a list of collected `ItemData` assets and calculates total stat boosts via `GetTotalBoost(StatType)`.

The `PlayerMovement` listens to `OnInventoryChanged` and calls `UpdateStats()`, reading the boost values to recalculate `groundMaxSpeed`, `airMaxSpeed` and `jumpForce`.

The terrain configuration also uses a `UpdatableData` base class for Scriptable Objects, allowing terrain parameters to propagate changes automatically in the editor via an `OnValuesUpdated` event, without requiring manual reloads.

This by the way caused me trouble building the game because `UnityEditor` is not available in builds and I had to wrap it.

Optimizations

I've wanted to implement occlusion culling as we saw during the course but the procedural nature of the terrain makes unity's static occlusion culling unusable.

Instead I used 3 important optimization strategies:

The first one is Distance-based culling. [EndlessTerrain.cs](https://endlessterrain.cs) keeps only chunks within maxViewDist visible, calling SetVisible(false) on any chunk outside of the view distance.

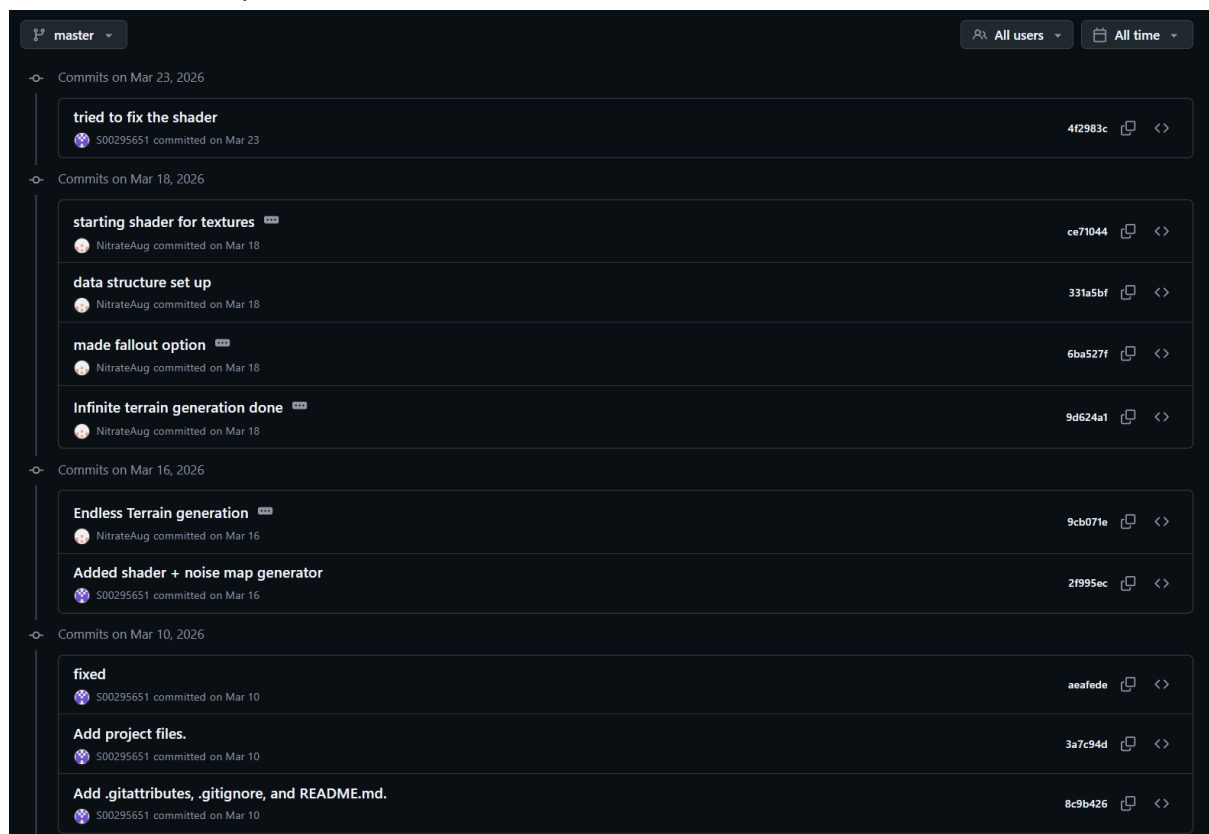
Next I used different levels of details, the further a chunk is to the player, the less polygon will be used and the less detailed the mesh will be..

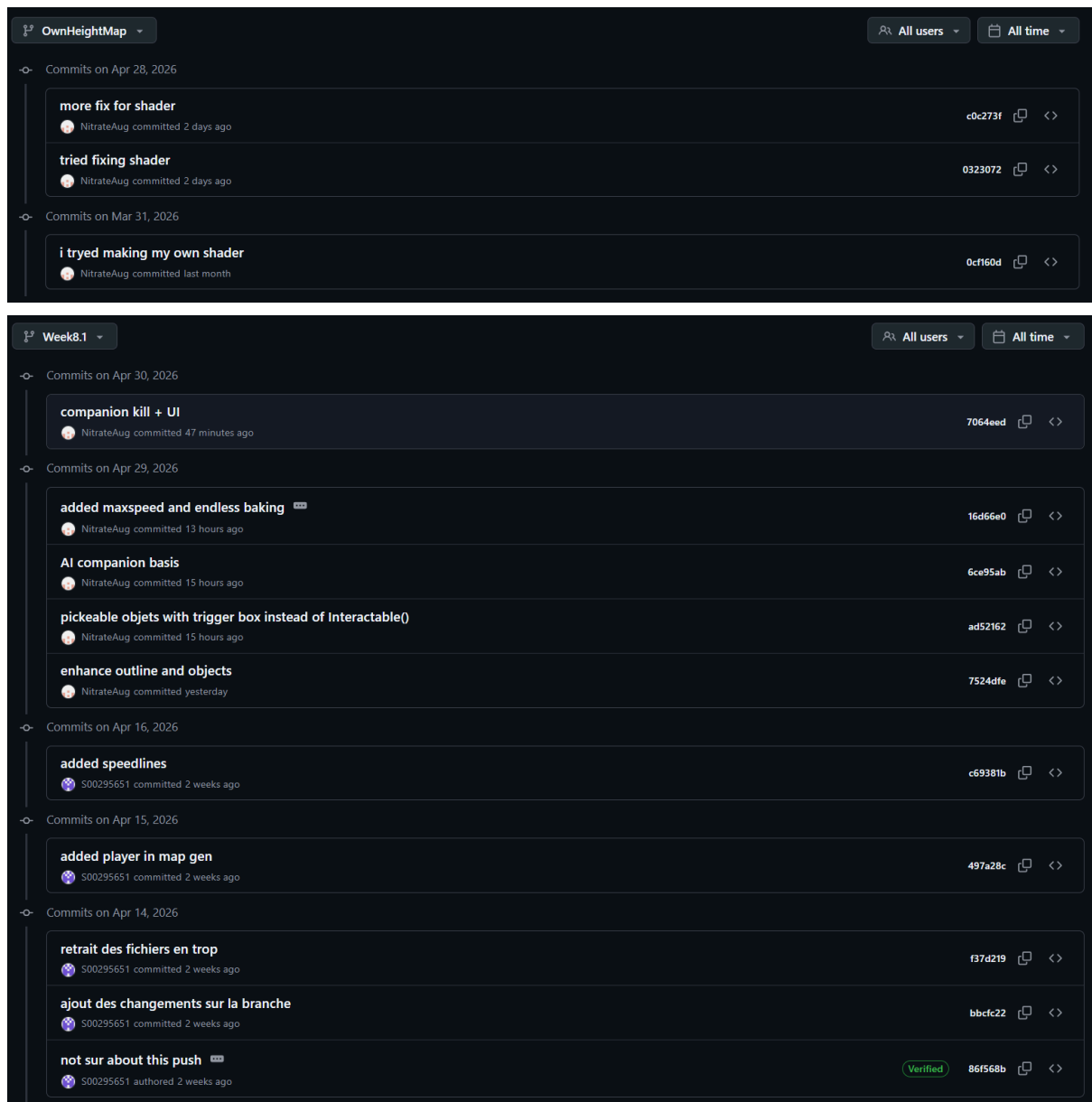
The last one is Frustum Culling, handled automatically by Unity

III. GitHub Development Process

Screenshot of commit history

For the procedural generation terrain I was following a unity tutorial unfortunately it was a 10 years old tutorial and it was not using URP so I had to improvise with the shader, that's why I made two more branches





IV. Future Improvements

What could be expanded or improved

This project was very interesting and I will continue working on it, currently the system of chaser is very simple and i'm not proud of my procedural generation :

Each chunk generates its mesh independently from its own MapData, and floating point precision differences in the noise sampling at the edges mean neighbouring chunks do not always match their vertex positions along their shared border.

Additionally, when two adjacent chunks are rendered at different LOD levels, their edge vertex counts do not match, creating visible cracks where the meshes don't align.